SQL - Basiskurs

Dipl.-Ing. Leonard Guelmino

Trainer

Dipl.-Ing. Leonard Guelmino

Trainer

E leonard@alpinecoders.dev

Über mich

Informatik @TU Wien

Trainer @WIFI Wien & @mit.at

CTO in einem Startup

Lieblingssprachen: Python, Typescript, Kotlin, Rust

Ehrenamt: Rotes Kreuz Korneuburg

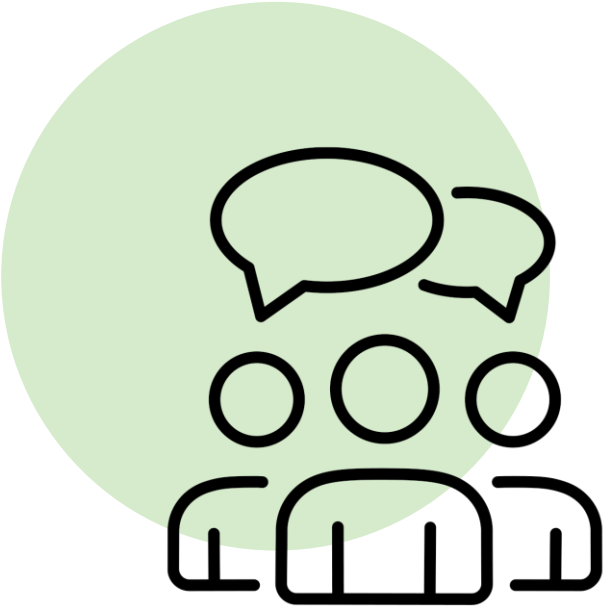
Hobby: Vogelkunde

Hallihallo!

Whoami?

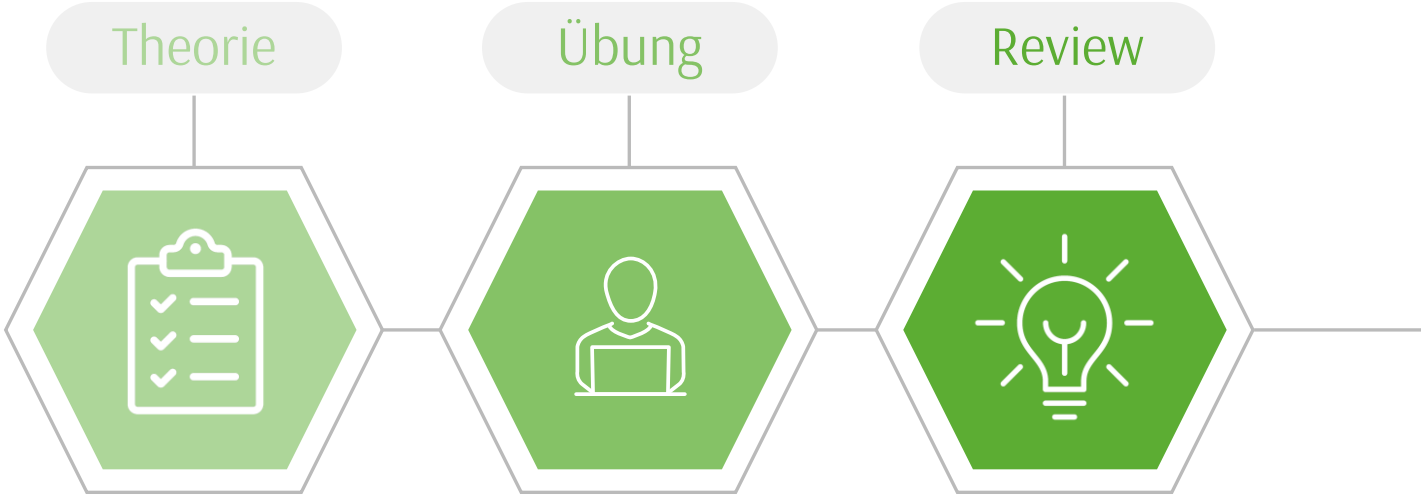
Welches Vorwissen bringe ich mit?

Was erwarte ich mir von diesem Kurs?



Methodik

20/70/10



Vortrag

Alle Lerninhalte werden zuerst theoretisch behandelt.

Aufgaben

Nach (fast) jedem Theoriemodul folgt eine Übungseinheit, in dem die Theoire praktisch angewandt wird.

Musterlösung

Jede Übung wird anhand der Musterlösung gemeinsam durchbesprochen und Fragen beantwortet.

Curriculum

Montag

VM

Modul 1: Einführung & Architektur

Modul 2: ERM & Datenbankdesign

NM

Modul 3: DDL – Datenbanken definieren

Modul 4: Erste Selektionen & Syntax-Regeln

Dienstag

VM

Modul 5: Daten-Setup & Filterung

Modul 6: Joins – Tabellen vereinigen

NM

Modul 7: Aggregation & Gruppierung

Modul 8: Sortierung & Formatierung

Mittwoch

VM

Modul 9: DML – Daten verwalten

Modul 10: Mengenoperationen & Subqueries

NM

Modul 11: Fortgeschrittene Konzepte

Kursplattform kennenlernen und technisches Setup überprüfen

Übung 0
Setup

Modul 1

Einführung & Architektur

AKH Wien

© C.Stadler/Bwag; CC-BY-SA-4.0

- 1.706 Betten
- 8.960 Angestellte (davon 1.631 Ärzt:innen)
- Behandlungen pro Jahr (2021):

  61.016 Stationär

  515.687 Ambulant
- Quelle: Wikipedia

Vom Papierarchiv zur relationalen Datenbank

Lfd-Nr	Pat_Vorname	Pat_Nachname	SVNummer	Arzt_Name	Arzt_Fachrichtung	Station_Name	Diagnose	Kosten
1	Max	Mustermann	1234010180	Dr. Gruber	Kardiologie	Herzstation	Hypertonie	150
2	Erika	Schmidt	5678020285	Dr. Gruber	Kardiologie	Herzstation	Check-up	80
3	Max	Mustermann	1234010180	Dr. Wagner	Neurologie	Neurologie West	MigrÃ¤ne	120
4	Max	Mustermann	1234010180	Dr. Gruber	Kardio-Logie	Herzstation	Kontrolle	90

Vom Papierarchiv zur relationalen Datenbank

- Datenmanagement im klinischen Umfeld erfordert **Konsistenz** und **Redundanzfreiheit**.
- Excel-Listen skalieren nicht für 1.500 Ärzte und Pflegekräfte im parallelen Schreibzugriff.
- Ein **Relationales Datenbankmanagementsystem (RDBMS)** erzwingt **Struktur** und garantiert **Datenintegrität**.
- SQL: **Structured Query Language**

Architektur: Motor vs. Lenkrad

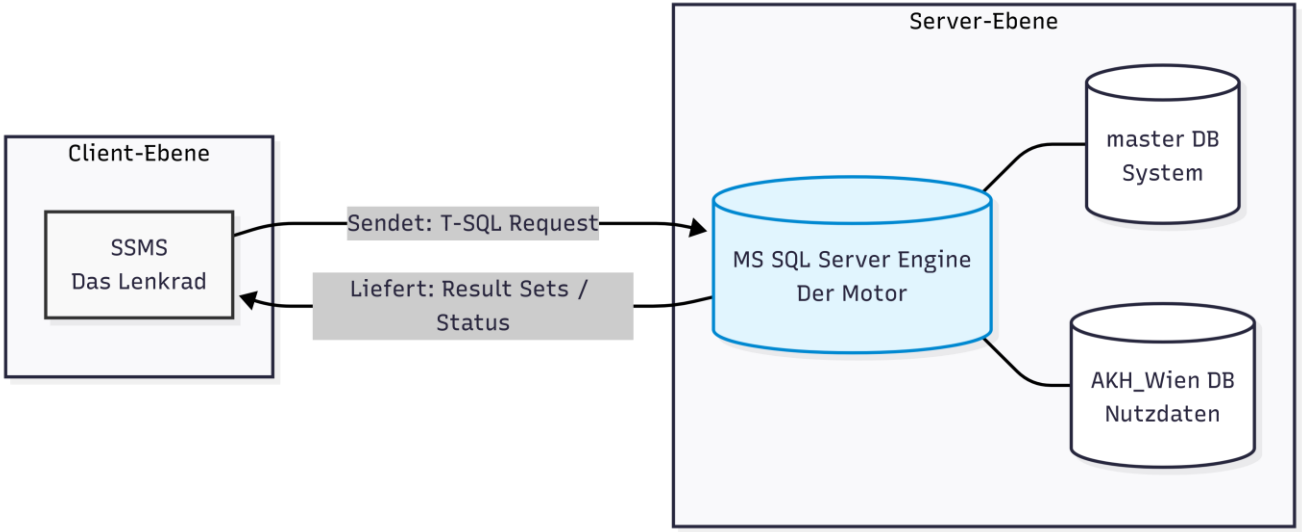
MS SQL Server & Management Studio (SSMS)

MS SQL Server (Der Motor):
Der Hintergrunddienst (sqlservr.exe). Er verarbeitet Daten, steuert Festplattenzugriffe und sichert Transaktionen.

SSMS (Das Lenkrad):
Die grafische Client-Anwendung. Sie sendet T-SQL-Befehle an den Server und visualisiert Ergebnisse.

Architektur: Motor vs. Lenkrad

MS SQL Server und Management Studio (SSMS)



Das Relationenmodell

Entitäten in Beziehung setzen

- Daten werden in strikt typisierten, zweidimensionalen Tabellen (**Entitäten**) strukturiert.
- Verknüpfungen zwischen Tabellen erfolgen über **logische Beziehungen**.

Das Relationenmodell

Entitäten in Beziehung setzen

- Die **Patienten**-Tabelle und die **Diagnosen**-Tabelle sind getrennt. Die Sozialversicherungsnummer (Primary Key) fungiert als universeller **Schlüssel** zur Verknüpfung.

Pat_ID	Vorname	Nachname	SVNummer
1	Max	Mustermann	1234010180
2	Erika	Schmidt	**5678020285**
3	Max	Mustermann	1234010180
4	Max	Mustermann	1234010180

Diagnose_ID	Arzt_Name	Arzt_Fachrichtung	Station_Name	Diagnose	SVNummer
1	Dr. Gruber	Kardiologie	Herzstation	Hypertonie	1234010180
2	Dr. Gruber	Kardiologie	Herzstation	sVT	**5678020285**
3	Dr. Wagner	Neurologie	Neurologie West	MigrÃ¤ne	1234010180
4	Dr. Gruber	Kardio-Logie	Herzstation	STEMI	1234010180

Datentypen

Text & Ganzzahlen

- **INT (Integer):** 4-Byte Ganzzahl. Hochperformant für System-IDs oder Stationsnummern.
- **CHAR(n):** Zeichenkette mit fixer Länge. Speichert Text effizient, da nur der tatsächlich benötigte Platz belegt wird (z.B. für SVNummer).
- **VARCHAR(n):** Zeichenkette variabler Länge (z.B. für Patientennamen).

Pat_ID	Vorname	Nachname	SVNummer
1	Max	Mustermann	1234010180
2	Erika	Schmidt	**5678020285**
3	Max	Mustermann	1234010180
4	Max	Mustermann	1234010180

Datentypen

Fließkommazahlen & Zeit

- **DECIMAL(p,s):** Fixkommazahl. Zwingend erforderlich für exakte Werte ohne Rundungsfehler. **DECIMAL(3,1)** speichert eine Körpertemperatur von 38.5 präzise.
- **DATETIME2 / DATE:** Chronologische Daten. Z.B. für Triage-Zeiten, Aufnahme, Entlassung und Medikamenten-Zyklen.

System-Initialisierung

Der erste Transact-SQL Befehl

Mit Data Definition Language (DDL) definieren wir die Systemstrukturen.

Wir trennen den logischen Arbeitsbereich explizit vom System-Kontext (master).

```sql
-- Erstellung der leeren Datenbank
CREATE DATABASE AKH_Wien;
GO

-- Kontextwechsel (verhindert Anlage in Systemdatenbanken)
USE AKH_Wien;
GO

-- Verifikation der Existenz (Aliasing ist Pflicht)
SELECT
    name AS DatabaseName,
    state_desc AS OperationalState
FROM sys.databases
WHERE name = 'AKH_Wien';
```

Modul 1 Stolperfallen

Best Practices

- Das Schließen des **SSMS** beendet den SQL Server *nicht*.
- Standardmäßig verbindet sich das SSMS mit der **master**-Datenbank. Ohne das Kommando **USE AKH_Wien** erstellen Sie Patienten-Tabellen versehentlich im kritischen Systembereich des Servers.
- Ein falsch gewählter Datentyp lässt sich im leeren System leicht ändern (**ALTER TABLE**). Bei 10 Millionen Patienten-Datensätzen führt dieselbe Änderung zu massiven Sperren (Table Locks) und Systemstillstand.

Modul 2

ERM & Datenbankdesign

Entitäten und Attribute

Das S in SQL

- **Entitäten** repräsentieren reale, abgrenzbare Informationsobjekte (z.B. Patient, Station).

- **Attribute** definieren die spezifischen, atomaren Eigenschaften vin Entitäten (z.B. Aufnahmedatum, Stationsname).

Primärschlüssel

Primary Key - PK

- Ein **PK** identifiziert jeden Datensatz einer Tabelle zweifelsfrei, eindeutig und unveränderlich.
- Auf Datenbankebene erzwingt der **PK** systemseitig die strikte Kombination der **UNIQUE** und **NOT NULL** Constraints.

Fremdschlüssel

Foreign Key - FK

- Ein **FK** in der Kind-Tabelle verweist zwingend auf den exakten **PK** der Eltern-Tabelle.
- Er garantiert **referentielle Integrität**. RDBMS blockieren Löschvorgänge, die Verweise ins Leere (Orphan Records) erzeugen würden.

Kardinalitäten

1:n Beziehung

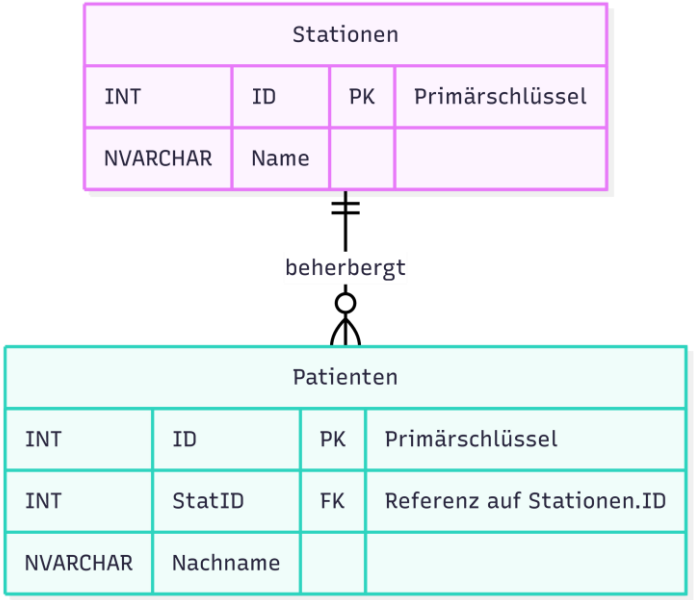
- **Kardinalitäten** definieren das quantitative Verhältnis zwischen zwei **Entitäten**.

- **1:n (One-to-Many):** Eine Station beherbergt zeitgleich **viele** Patienten, aber ein Patient liegt auf **exakt einer** Station.

Kardinalitäten

1:n Beziehung

- Der **Fremdschlüssel** liegt im RDBMS zwingend immer auf der "**n**"-Seite.



Kardinalitäten

m:n Beziehung

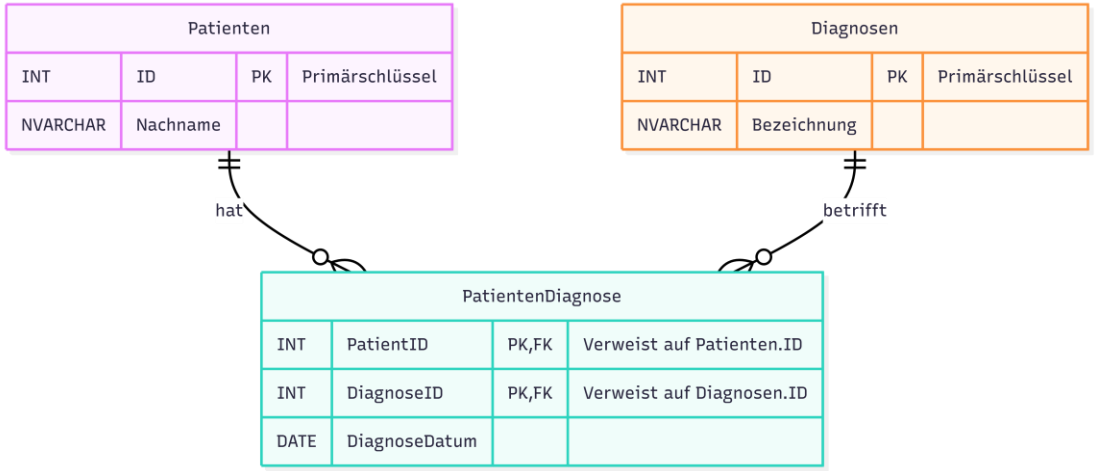
- **m:n (Many-to-Many):** Ein Patient hat im Lebenszyklus mehrere Diagnosen; eine spezifische Diagnose betrifft viele Patienten.

- **RDBMS können m:n-Beziehungen physisch nicht abbilden**. Sie erfordern eine zwingende Auflösung durch eine Mapping-Tabelle.

Kardinalitäten

m:n Beziehung

- Die Tabelle **PatientenDiagnose** speichert die Kombination aus **PatientID** und **DiagnoseID** und wandelt die Struktur in **zwei 1:n** Beziehungen um.



Kardinalitäten

1:1 Beziehung

- **1:1 (One-to-One):**.Ein Patient hat eine E-card. Eine E-card gehört einem Patienten.

- SVNR wird nicht in eine eigene Tabelle geschrieben

Praxis: SSMS Database Diagram Tool

Visuelle Modellierung vor der DDL-Phase

- Das Tool ermöglicht die grafische Konstruktion von **Tabellen**, **Datentypen** und **FK-Beziehungen** direkt in der Metadaten-Struktur der Datenbank.
- Validierung der relationalen Logik erfolgt visuell, bevor fehlerhafter DDL-Code manuell geschrieben wird.
- Änderungen im Diagramm generieren im Hintergrund automatisiert das korrekte **T-SQL-Skript** für die Schema-Anpassung.

Modul 2 Stolperfallen

Best Practices

⚠️ Tipp: Natürliche vs. Künstliche Schlüssel

- Verwenden Sie für **Primärschlüssel** in der Praxis immer künstliche Schlüssel wie **INT IDENTITY(1,1)**.
- **Natürliche Schlüssel** scheitern an der Realität: Ein bewusstloser Notfallpatient (John Doe) hat initial keine SVN. Da ein **PK** aber zwingend **NOT NULL** erfordert, würde das System die Aufnahme des Datensatzes technisch verweigern.

ERM im SSMS

Übung 1
ERM & Datenbankdesign

Modul 3

DDL

Die Geburtsstunde der Datenbank

Vom Modell zur physischen Struktur

- **Data Definition Language (DDL)** übersetzt das visuelle **ER-Modell** in persistente Systemstrukturen.

- **CREATE TABLE** reserviert Speicherplatz und erzwingt das exakte Metadaten-Schema im SQL Server.

CREATE TABLE: Stammdaten Station

Aufbau der referenzierten Entität (Eltern-Tabelle)

- Relationales Design diktiert die Erstellungsreihenfolge: Entitäten ohne Fremdschlüssel müssen zwingend zuerst instanziiert werden.

- Jeder Spalte wird ein strikter Datentyp und ein Nullability-Zustand zugewiesen.

```sql
CREATE TABLE Station (
    StatID INT IDENTITY(1,1) PRIMARY KEY,
    StationName VARCHAR(50) NOT NULL,
    Stockwerk TINYINT NOT NULL
);
```

CREATE TABLE: Entität Patient

Fremdschlüssel und fixe Längen (Kind-Tabelle)

Die Definition der Kind-Tabelle inkludiert die harte, physische FK-Beziehung zur Tabelle Station.

Der Datentyp CHAR(10) garantiert die exakte, unveränderliche Länge der österreichischen Sozialversicherungsnummer (SVN).

Dynamische Längen (VARCHAR) sind hier technisch ineffizient.

```sql
CREATE TABLE Patient (
    PatID INT IDENTITY(1,1) PRIMARY KEY,
    Nachname VARCHAR(50) NOT NULL,
    Vorname VARCHAR(50) NOT NULL,
    GebDatum DATE NOT NULL,
    SVNummer CHAR(10) UNIQUE,
    StatID INT FOREIGN KEY REFERENCES Station(StatID)
);
```

ALTER & DROP: Laufende Anpassungen

Operationen am offenen System

ALTER TABLE mutiert bestehende Tabellenstrukturen (Spalten-Addition, Datentyp-Wechsel) ohne Datenverlust im laufenden Betrieb.

DROP TABLE vernichtet die Tabellenstruktur mitsamt aller darin enthaltenen Daten irreversibel auf Festplattenebene.

```sql
-- Erweiterung der Patientenakte um eine optionale Kontaktmöglichkeit
ALTER TABLE Patient
ADD Telefonnummer VARCHAR(20) NULL;

-- Irreversible Löschung (Vorsicht!)
-- DROP TABLE Station;
```

Constraints

Regelwerke auf Datenbankebene

- **DEFAULT**: Vergabe eines Standardwertes.

```sql
CREATE TABLE Behandlungen (
    ID INT IDENTITY(1,1) PRIMARY KEY,
    -- ....
    Status NVARCHAR(50) DEFAULT 'Abgeschlossen' -- <--- DEFAULT Constraint
);
```

Constraints

Regelwerke auf Datenbankebene

- **NOT NULL**: Verhindert unvollständige Datensätze (z.B. ein Patient *muss* zwingend ein Geburtsdatum haben).

```
ALTER TABLE Patienten
ALTER COLUMN GebDatum DATE NOT NULL;
```

Constraints

Regelwerke auf Datenbankebene

- **UNIQUE**: Garantiert systemweite Eindeutigkeit für Nicht-PK-Attribute. Die SVN darf nur exakt einmal existieren.

```
ALTER TABLE Patienten
ADD CONSTRAINT UQ_Patienten_SVNummer UNIQUE (SVNummer);
```

Constraints

Regelwerke auf Datenbankebene

- **CHECK**: Validiert Wertebereiche direkt beim Schreibvorgang um Tippfehler zu verhindern.

```
ALTER TABLE Stationen
ADD CONSTRAINT CHK_Stationen_Stockwerk CHECK (Stockwerk >= 0 AND Stockwerk <= 20);
```

Modul 3 Stolperfallen

Best Practices

⚠️ Achtung: Referentielle Integrität blockiert Zerstörung

- Führen Sie **DROP TABLE Station;** aus, wird der SQL Server dies mit einer Fehlermeldung quittieren, sofern Datensätze in der Tabelle Patient existieren.
- Das **RDBMS** schützt sich selbst: Eine **Station** kann nicht abgerissen werden, solange **Patienten** darauf **referenzieren**. Zuerst müssen die Patienten verlegt (**geändert**) oder entlassen (**gelöscht**) werden.

Skripting & Ausführung im SSMS

Übung 2
DDL

Modul 4

Erste Datenabfragen & Syntax-Regeln

Dateneingabe

INSERT INTO ‹Tabelle› (‹Attributname›) VALUES (‹Attributwert›)

INSERT INTO injiziert physische Datensätze in die zuvor definierte Tabellenstruktur.

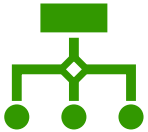
Das RDBMS validiert jeden Datensatz in Echtzeit strikt gegen die definierten Constraints (NOT NULL, UNIQUE, FOREIGN KEY).

Ein Verstoß (z.B. die Eingabe einer bereits existierenden Sozialversicherungsnummer) führt zu einem **Rollback** der gesamten **Transaktion**.

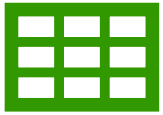
```
-- Anlage der Eltern-Entität (Station)
INSERT INTO Station (StationName, Stockwerk)
VALUES ('Kardiologie', 3);

-- Anlage der Kind-Entität mit Referenz (StatID = 1)
INSERT INTO Patient (Nachname, Vorname, GebDatum, SVNummer, StatID)
VALUES ('Müller', 'Anna', '1985-05-20', '1234200585', 1);
```

Datenabfrage

SELECT ‹Attribut› FROM ‹Tabelle›



SELECT definiert exakt, *welche* Spalten (Attribute) das Result Set (die Ergebnismenge) enthalten soll.



FROM spezifiziert die exakte Quelle (die physische Tabelle), aus der die Daten gelesen werden.

```
SELECT
    SVNummer AS Sozialversicherungsnummer,
    Nachname AS Familienname
FROM Patient;

```

Aliasing und Literale

SELECT ‘<Literal>‘ AS <Alias> FROM <Tabelle>

- **Spaltennamen** im System (z.B. GebDatum) sind aus Gründen der Namenskonvention für Endanwender oft kryptisch.
- Das Schlüsselwort **AS** erzeugt einen temporären Spalten-**Alias**, der ausschließlich für die finale Anzeige gerendert wird.
- **Literale** (fest codierte Text- oder Zahlenwerte) reichern das Result Set um statische Informationen an, die nicht in der Tabelle existieren.

```sql
SELECT
    'AKH Wien' AS Klinikstandort,
    Nachname AS PatientenNachname,
    Vorname AS PatientenVorname
FROM Patient;
```

Berechnungen zur Laufzeit

Dynamische Informationsgewinnung

- **T-SQL** kann mathematische Operationen **(+, -, *, /)** oder **Datumsfunktionen** direkt in der **SELECT**-Klausel ausführen.
- Berechnete Spalten existieren nur im flüchtigen RAM (**Result Set**) und belegen keinen Speicherplatz auf der Festplatte.

```sql
SELECT
    Nachname AS Familienname,
    YEAR(GETDATE()) - YEAR(GebDatum) AS GeschaetztesAlter
FROM Patient;
```

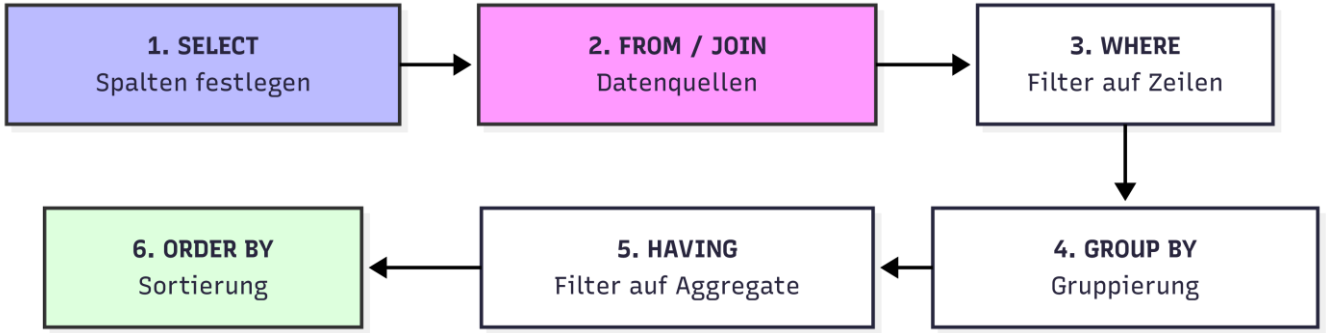
Die Engine denkt anders als wir schreiben

Lexikalische vs. Logische Verarbeitungsreihenfolge

- Code wird lexikalisch *geschrieben* (**SELECT -> FROM -> WHERE**), aber von der Engine logisch abweichend *ausgeführt*.
- Die Ausführung startet zwingend bei **FROM** (Woher kommen die Daten?), bevor **SELECT** (Was brauche ich davon?) greift.
- **Konsequenz:** Wenn wir in Filter setzen (WHERE), kennt der Server die in der SELECT-Klausel definierten Aliase technisch noch gar nicht. Ein Filter auf **WHERE GeschaetztesAlter > 50** wirft daher unweigerlich einen Syntaxfehler.

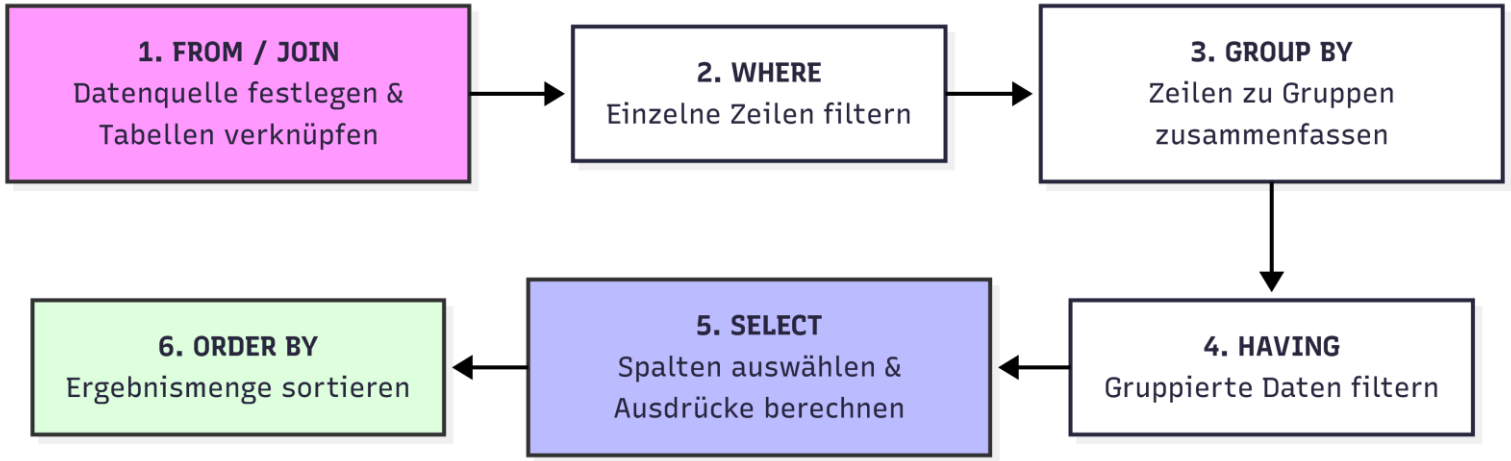
T-SQL

Lexikalische Reihenfolge



T-SQL

Processing Reihenfolge



Modul 4 Stolperfallen

Best Practices

⚠️ Achtung: Der SELECT * Anti-Pattern

- Ein **SELECT *** (Gib mir alle Spalten) ist in der Produktionsumgebung strengstens verboten.
- Es zerschießt die Applikationsobjekte (wenn nachträglich neue Spalten in der Tabelle hinzugefügt werden) und verschwendet massiv Netzwerkbandbreite, da Spalten übermittelt werden, die das Frontend womöglich gar nicht benötigt.
- Schreiben Sie Spalten in produktivem Code immer **explizit** aus.

Übung 3
SELECT & Syntax-Regeln

Modul 5

Daten-Setup & Filterung

Daten-Setup: Das AKH-System-Backup

Vom leeren System zur realen Datenlast

- Ein **RESTORE DATABASE**-Vorgang überschreibt die aktuelle Struktur mit einem physischen Snapshot (der .bak-Datei).
- Es ist der kritische Reset-Button nach einem Datenverlust und transferiert vorbereitete Entitäten (Patienten, Behandlungen, ...) in unser Testsystem.

```sql
-- Exklusiver Zugriff zwingend erforderlich
USE master;
GO

RESTORE DATABASE AKH_Wien
FROM DISK = 'C:\Backups\AKH_Wien_Full.bak'
WITH REPLACE;
GO
```

Filterung mit WHERE

... WHERE ‹Attribut› ‹Operator› ‹Wert›

- Die **WHERE**-Klausel greift logisch *vor* dem **SELECT**. Sie filtert Datensätze direkt beim Lesen von der Festplatte.

- **Vergleichsoperatoren (=, <>, >, <)** definieren strikte, boolesche Bedingungen (**Wahr** oder **Falsch**).

```sql
SELECT
    BehandlungsID AS Fallnummer,
    Kosten AS Behandlungskosten_EUR
FROM Behandlung
WHERE Kosten >= 5000.00;
```

Logische Verknüpfungen

AND & OR

- **AND** erzwingt zwingend die Erfüllung *aller* verknüpften Bedingungen (Schnittmenge).
- **OR** erfordert die Erfüllung von *mindestens einer* Bedingung (Vereinigungsmenge).
- **Klammern ()** sind bei gemischten Ausdrücken Pflicht. Die Engine priorisiert **AND** technisch **höher** als **OR**.

```sql
SELECT
    PatID AS PatientenNummer,
    Datum AS Behandlungsdatum
FROM Behandlung
WHERE (ArztID = 12 OR ArztID = 15)
  AND Kosten > 1000.00;
```

Bereichsfilter

IN & BETWEEN

- **IN (x, y, z)** ersetzt unübersichtliche **OR**-Ketten auf derselben Spalte. Es ist syntaktisch sauberer und für den Query Optimizer leichter zu parsen.
- **BETWEEN x AND y** prüft geschlossene Intervalle (inklusive der definierten Grenzen). Es ist der Standard für chronologische oder monetäre Auswertungen.

```sql
SELECT
    PatID AS PatientenNummer,
    Kosten AS Behandlungskosten_EUR
FROM Behandlung
-- Entspricht: WHERE Kosten >= 1000.00 AND Kosten <= 5000.00
WHERE Kosten BETWEEN 1000.00 AND 5000.00;
```

NULL

Das Konzept des fehlenden Wertes

- **NULL** ist auf Datenbankebene niemals die Ziffer Null (0) und niemals ein leerer String (‚‘).
- **NULL** bedeutet **"Unbekannt"** oder **"Derzeit nicht anwendbar"**.

```sql
SELECT
    PatID AS PatientenNummer,
    ArztID AS BehandelnderArzt
FROM Behandlung
-- FALSCH: WHERE ArztID = NULL
WHERE ArztID IS NULL;
```

Modul 5 Stolperfallen

Best Practices

⚠️ Achtung: Die BETWEEN-Falle bei Datumsfeldern

- Wird **BETWEEN '2023-01-01' AND '2023-12-31'** auf eine **DATETIME-Spalte** angewendet, filtert die Engine exakt bis **2023-12-31 00:00:00.000**.
- Eine OP am **31.12. um 08:00** Uhr morgens fällt unsichtbar aus dem Report!
- Nutzen Sie bei Zeitstempeln **offene Intervalle**: **WHERE Datum >= '2023-01-01' AND Datum < '2024-01-01'**.

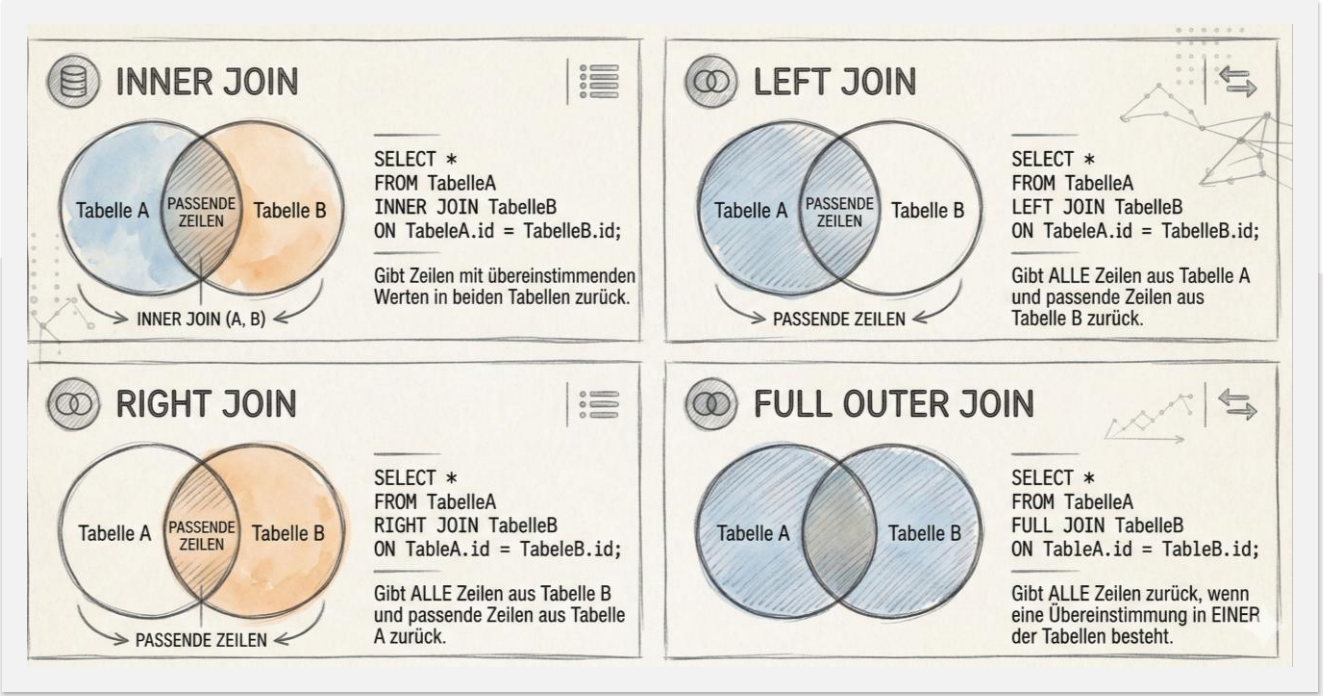
Übung 5
Filterung

Modul 6 | Joins

Joins: Das Herz relationaler Datenbanken

Schnittmengen und Datenfusion

- RDBMS zerlegen Daten zur Vermeidung von Redundanz in Einzeltabellen (**Normalisierung**).
- **Joins** setzen diese dezentralen Fragmente zur Laufzeit wieder zu einer lesbaren, zweidimensionalen Tabelle zusammen.
- Die Verknüpfung erfordert zwingend eine logische Übereinstimmung – in der Regel das exakte Matching von **Primary Key** und **Foreign Key**.



Joins: Überblick

Exakte Schnittmenge

INNER JOIN

Exakte Schnittmenge

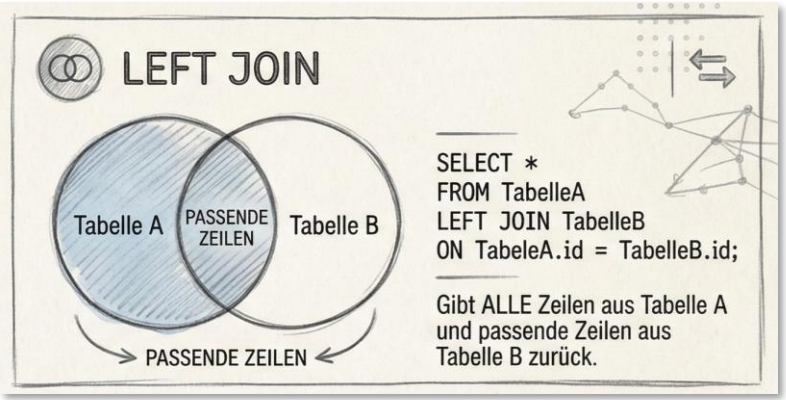
- Der **INNER JOIN** retourniert ausschließlich Datensätze, bei denen die definierte **ON**-Bedingung in ***beiden* Tabellen** erfüllt ist.
- Datensätze ohne Pendant (z. B. eine brandneue Station ohne Patienten) fallen zwingend aus dem Result Set.

```sql
SELECT
    p.Nachname AS PatientenName,
    p.SVNummer AS Sozialversicherungsnummer,
    s.StationName AS ZugewieseneStation
FROM Patient AS p
INNER JOIN Station AS s
    ON p.StatID = s.StatID;
```

LEFT JOIN

Inklusion von Randmengen

- Der **LEFT OUTER JOIN** erzwingt die Ausgabe *aller* Datensätze der **linken Tabelle** (die nach dem FROM steht), unabhängig von einem Treffer rechts.
- Fehlt der entsprechende Datensatz auf der rechten Seite, füllt die SQL Server Engine die projizierten Spalten systemseitig mit NULL auf.



OUTER JOIN

Anomalien Aufdecken

- Die Kombination aus LEFT JOIN und WHERE ... IS NULL ist das Standardwerkzeug von Datenanalysten, um verwaiste Datensätze ("Sorgenkinder") zu identifizieren.
- Wir suchen gezielt nach der Abwesenheit einer Referenz.

```sql
SELECT
    p.Nachname AS UnbehandelterPatient,
    p.SVNummer AS Sozialversicherungsnummer,
    b.BehandlungsID AS Fallnummer
FROM Patient AS p
LEFT JOIN Behandlung AS b
    ON p.PatID = b.PatID
WHERE b.BehandlungsID IS NULL;
-- Resultat: Liste aller Patienten, die noch nie eine Behandlung erhalten haben.
```

SELF-JOIN

Hirarchische Srtukturen sichtbar machen

- Ein Join einer Tabelle mit sich selbst erfordert zwingend unterschiedliche Tabellen-Aliase (a1, a2).
- Dient der Abbildung monohierarchischer Strukturen innerhalb derselben Entität (z. B. ein Arzt referenziert auf einen anderen Arzt).

```sql
SELECT
    Assistenz.Nachname AS Turnusarzt,
    Oberarzt.Nachname AS Vorgesetzter
FROM Arzt AS Assistenz
LEFT JOIN Arzt AS Oberarzt
    ON Assistenz.VorgesetzterID = Oberarzt.ArztID;
```

Modul 6 Stolperfallen

Best Practices

⚠️ Achtung: Das Kartesische Produkt (CROSS JOIN)

- Vergessen Sie bei einem Join die **ON-Bedingung,** multipliziert der SQL Server jeden Datensatz der linken Tabelle mit jedem der rechten Tabelle.
- 1.500 Patienten * 50 Stationen = 75.000 generierte Zeilen im RAM.
- Dies führt bei echten Produktionsdatenmengen augenblicklich zur Überlastung der TempDB und zum Absturz der Applikation. Verwenden Sie immer explizite JOIN-Syntax.

Übung 6
Joins

Modul 7

Aggregation & Gruppierung

Aggregatfunktionen

Kennzahlen für Datenanalyse generieren

- **Aggregation** komprimiert Millionen Einzeldatensätze verlustfrei auf eine **einzige**, hochverdichtete Zeile.
- Mathematische Standardfunktionen **(SUM, AVG, MIN, MAX)** operieren vertikal auf der kompletten Datenspalte.

COUNT

Physische Zeilen vs. Werte

- **COUNT(*)** zählt gnadenlos jede physisch vorhandene Zeile im Result Set, unabhängig von ihrem Inhalt.
- **COUNT(Spaltenname)** zählt exklusiv bekannte Werte und **ignoriert** NULL-Zustände rigoros.
- Von 100 Patienten haben nur 80 eine Telefonnummer hinterlegt. **COUNT(*)** liefert **100**. **COUNT(Telefonnummer)** liefert **80**.

```sql
SELECT
    COUNT(*) AS AnzahlPatientenGesamt,
    COUNT(Krankenkasse) AS AnzahlVersichertePatienten
FROM Patient;
```

GROUP BY

Kategorisierung von Datenmengen

- **GROUP BY** zerlegt das globale Result Set in logische, in sich geschlossene Teilmengen (z.B. Aggregation pro Station).
- Die eiserne Engine-Regel: Jede Spalte im **SELECT** muss zwingend entweder aggregiert **(SUM, COUNT)** oder im **GROUP BY** deklariert sein.

```sql
SELECT
    Krankenkasse AS Versicherungstraeger,
    COUNT(PatID) AS AnzahlPatienten
FROM Patient
GROUP BY Krankenkasse;
```

GROUP BY

Mehrdimensionale Aggregation

- **Gruppierungen** über mehrere Spalten erzeugen eine feingranulare Daten-Matrix (z.B. Kosten pro Arzt und Jahr).

- Die lexikalische Reihenfolge der Spalten im **GROUP BY** diktiert die interne Gruppierungshierarchie des Servers.

```sql
SELECT
    ArztID AS BehandelnderArzt,
    YEAR(Datum) AS Abrechnungsjahr,
    SUM(Kosten) AS GesamtKosten_EUR
FROM Behandlung
GROUP BY
    ArztID,
    YEAR(Datum);
```

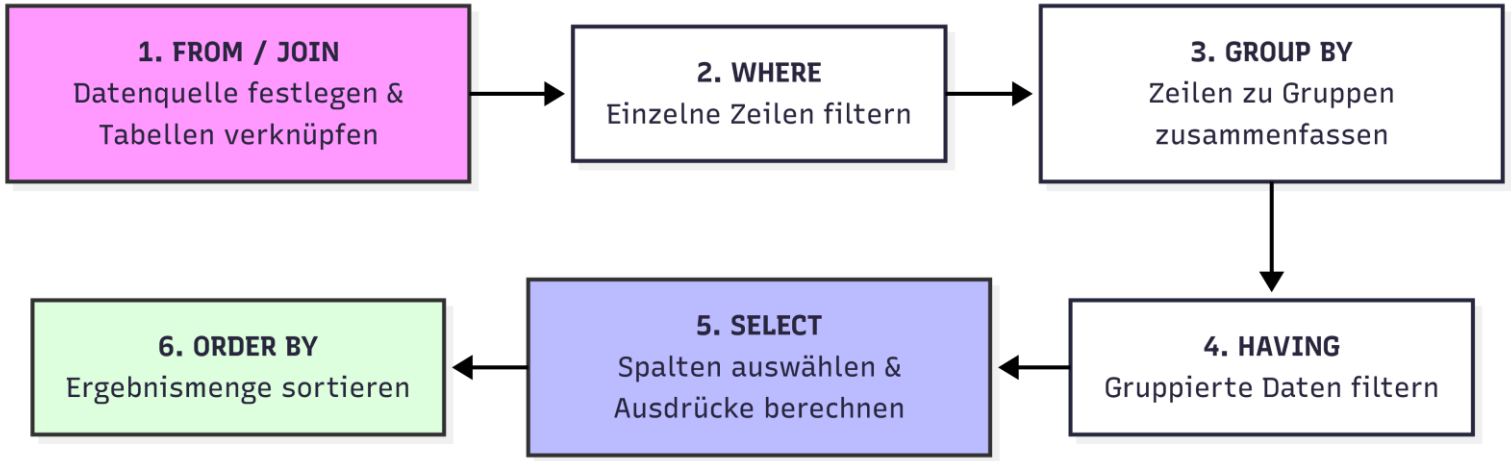
HAVING

Filterung NACH der Aggregation

- **WHERE** filtert atomare Zeilen ***vor*** der mathematischen Aggregation (Festplatte).
- **HAVING** filtert ausschließlich konsolidierte Gruppen ***nach*** der Aggregation (RAM).

```sql
SELECT
    ArztID AS BehandelnderArzt,
    SUM(Kosten) AS ErwirtschafteterUmsatz_EUR
FROM Behandlung
GROUP BY ArztID
HAVING SUM(Kosten) > 50000.00;
```

T-SQL Processing Reihenfolge



Modul 7 Stolperfallen

Best Practices

⚠️ **Achtung: WHERE und HAVING niemals verwechseln**

- Ein Filter auf **WHERE SUM(Kosten) > 5000** ist syntaktisch unmöglich. Die Engine kann keine Summe filtern, die in der logischen Verarbeitungsreihenfolge noch gar nicht berechnet wurde.
- **Performance:** Filtern Sie so viele Daten wie möglich frühzeitig im **WHERE** (z.B. nur das aktuelle Jahr). Riesige Datenmengen zu aggregieren, nur um 90% davon per **HAVING** wieder zu verwerfen, verschwendet CPU-Zyklen.

Übung 6
Aggregation & Gruppierung

Modul 8

Sortierung & Formatierung

ORDER BY

Sortierung von Result Sets

- Ohne explizites **ORDER BY** garantiert der SQL Server **keinerlei** Sortierreihenfolge. Die Ausgabe basiert auf dem zufälligen physischen Lesezugriff der Engine.
- Die Sortierung erfolgt ganz am Ende der logischen Verarbeitungskette (nach der SELECT-Klausel). Daher kann im ORDER BY auf die Aliase zugegriffen werden.

ORDER BY

Mehrdimensionale Sortierung

- **Sortierung über mehrere Spalten** erlaubt feingranulare Listen (**ASC** für aufsteigend, **DESC** für absteigend).

- Die **lexikalische Reihenfolge** der Spalten im Code definiert zwingend die Priorität der Sortier-Gewichtung.

```sql
SELECT
    StatID AS StationsNummer,
    Nachname AS Familienname,
    Vorname AS Rufname
FROM Patient
ORDER BY
    StatID ASC,
    Nachname ASC,
    Vorname ASC;
```

CONCAT & SUBSTRING

Textmanipulation zur Laufzeit

- **CONCAT()** verbindet isolierte String-Fragmente robust und wandelt **NULL-Werte** implizit in **leere Strings** um (im Gegensatz zum + Operator).
- **SUBSTRING()** extrahiert definierte Zeichenketten-Teile, primär genutzt. Z.B. für anonymisierte Patienten-Kürzel oder Auswertungen von Diagnoseschlüsseln.

```sql
SELECT
    CONCAT(Nachname, ', ', Vorname) AS Vollname,
    SUBSTRING(SVNummer, 1, 4) AS Geburtsjahrgang_SVN
FROM Patient;
```

GETDATE & DATEDIFF

Chronologische Berechnungen

- **GETDATE()** liest den exakten, aktuellen Systemzeitstempel des Betriebssystems aus.

- **DATEDIFF(Einheit, Start, Ende)** berechnet die harte mathematische Differenz zwischen zwei Zeitpunkten (z.B. in Tagen dd oder Stunden hh).

```sql
SELECT
    BehandlungsID AS Fallnummer,
    DATEDIFF(dd, Datum, GETDATE()) AS TageSeitBehandlung
FROM Behandlung
WHERE Datum IS NOT NULL;
```

Datumsformatierung

TT.MM.JJJJ

- Interne **DATETIME**-Werte haben auf der Festplatte kein Format.
- Das **T-SQL** Spezifikum **CONVERT(VARCHAR, Datum, 104)** erzwingt den deutschen/österreichischen Industriestandard.

```sql
SELECT
    Nachname AS Familienname,
    CONVERT(VARCHAR(10), GebDatum, 104) AS Geburtsdatum_AT,
    FORMAT(GebDatum, 'dd.MM.yyyy') AS Geburtsdatum_Langsam
FROM Patient;
```

Modul 8 Stolperfallen

Best Practices

⚠️ Achtung: FORMAT() vs. CONVERT() – Eine Performance-Falle

- Seit SQL Server 2012 existiert die **FORMAT()**-Funktion (basiert auf der .NET CLR). Sie ist extrem flexibel, aber **CPU-intensiv**.
- Bei 100 Datensätzen ist das irrelevant. Bei einem Report über 2 Millionen Behandlungen zwingt **FORMAT()** den SQL Server in die Knie.
- Nutzen Sie für Reports auf großen Datenmengen ausschließlich das effiziente **CONVERT()**.

Übung 7
Sortierung & Formattierung

Modul 9

DML
Daten verwalten

Data Manipulation Language (DML)

Schreibende Operationen im System

- **Data Manipulation Language (DML)** steuert den Lebenszyklus der Datensätze (**INSERT, UPDATE, DELETE**).
- Im Gegensatz zu **DQL (SELECT)** verändern **DML**-Operationen den physischen Zustand der Speicherseiten (Data Pages) **dauerhaft**.

INSERT INTO

Anlage neuer Datensätze

- Injiziert neue Tupel (Zeilen) präzise in eine bestehende Tabellenstruktur.
- Die übergebene Spaltenliste muss zwingend exakt mit der Datentyp-Reihenfolge der VALUES-Liste korrelieren.
- Das Fehlen von DEFAULT-Const Transaktion.

```
INSERT INTO Patient (Nachname, Vorname, GebDatum, SVNummer, StatID)
VALUES ('Huber', 'Josef', '1965-11-12', '4567121165', 2);
```

UPDATE

Gezielte Modifikation von Attributen

- **Überschreibt existierende Werte** in spezifischen Spalten mittels der **SET-Klausel**.
- Betrifft systemseitig ***alle*** Datensätze der Tabelle, sofern die Menge nicht explizit **durch Bedingungen eingeschränkt** wird.

```
UPDATE Patient
SET StatID = 5 -- 5 = Intensivstation (PK-Referenz)
WHERE SVNummer = '4567121165';
```

DELETE

Physische Löschung auf Zeilenebene

- **Entfernt** spezifische Datensätze aus einer Tabelle. Jeder gelöschte Datensatz wird im Transaction Log einzeln protokolliert (Row-by-Row).
- Strikte Limitierung: Referentielle Integrität (**FOREIGN KEY**) **blockiert** den Löschvorgang rigoros, sobald abhängige Kind-Datensätze (z.B. Behandlungen) existieren.

```sql
DELETE FROM Patient
WHERE SVNummer = '0000000000'; -- Dummy-Eintrag sicher entfernen
```

DELETE vs. TRUNCATE

Zeilenlöschung vs. System-Reset

- **DELETE:** DML-Operation. Verarbeitet Zeile für Zeile. Langsam, aber präzise und löst Trigger aus.
- **TRUNCATE:** DDL-Operation. Deallokiert die physischen Speicherseiten der Tabelle auf einen Schlag und setzt den IDENTITY-Zähler (Auto-Increment) auf den Startwert zurück.
- **TRUNCATE** ist bei Millionen von Datensätzen extrem **performant**. Es scheitert jedoch sofort an Tabellen, die von einem **Fremdschlüssel** referenziert werden – unabhängig davon, ob die Kind-Tabelle Daten enthält oder nicht.

Modul 9 Stolperfallen

Best Practices

⚠️ Achtung: Update ohne WHERE

- Ein **UPDATE Patient SET StatID = NULL;** ohne **WHERE**-Klausel entlässt augenblicklich alle 1.500 Patienten des AKH Wien aus ihren Betten.
- Der Befehl ist **syntaktisch korrekt** und wird von der SQL Server Engine anstandslos ausgeführt.
- **Prävention:** Schreiben Sie zwingend ***immer*** zuerst ein **SELECT** mit der **WHERE**-Klausel, um die betroffene Datenmenge visuell zu verifizieren. Ersetzen Sie erst danach das **SELECT** durch **UPDATE** oder **DELETE**.

Übung 8
DML

Modul 10

Mengenoperationen & Subqueries

Vertikale Datenfusion

Mengenoperationen: Strukturierte Stapelverarbeitung

- Im Gegensatz zu **Joins** (horizontale Erweiterung) stapeln **Mengenoperationen** Result Sets **vertikal** aneinander.
- Zwingende Systemvoraussetzung: Exakt gleiche Anzahl an Spalten und kompatible Datentypen in beiden SELECT-Statements.

UNION (ALL)

Performance und Duplikate

- **UNION** erzwingt im Hintergrund eine rechenintensive **Sortieroperation** (implizites **DISTINCT**), um Zeilen-Duplikate zu identifizieren und zu verwerfen.
- **UNION ALL** hängt die Speicherblöcke blind und ohne Prüfung aneinander. Dies schont die CPU und ist bei **disjunkten Mengen** (wie Aktiv vs. Archiv) zu verwenden.

```sql
SELECT
    SVNummer AS Sozialversicherungsnummer,
    Nachname AS Familienname
FROM Patient

UNION ALL

SELECT
    SVNummer AS Sozialversicherungsnummer,
    Nachname AS Familienname
FROM Patient_Archiv;
```

Subqueries

Dynamische Filterkriterien zur Laufzeit

- Eine **Subquery** (Unterabfrage) ist ein vollwertiges **SELECT**-Statement, das gekapselt in runden Klammern **() innerhalb einer Hauptabfrage** operiert.
- Die Engine **evaluiert die innere Abfrage autark** und übergibt das mathematische Resultat an das äußere Statement.

Subqueries

Komplexe WHERE-Bedingungen

- Hardcodierte Schwellenwerte veralten im Klinikbetrieb schnell. **Subqueries berechnen Vergleichswerte dynamisch auf Basis des aktuellen Systemzustands**.
- Liefert die **Subquery** das Kriterium für einen **Operator** wie >, <, oder =, muss sie zwingend einen **Skalarwert** (genau eine Zeile und eine Spalte) zurückgeben.

```sql
SELECT
    BehandlungsID AS Fallnummer,
    PatID AS PatientenNummer,
    Kosten AS Behandlungskosten_EUR
FROM Behandlung
WHERE Kosten > (
    SELECT AVG(Kosten) AS DurchschnittlicheKosten
    FROM Behandlung
);
```

Modul 10 Stolperfallen

Best Practices

⚠️ Achtung: Multi-Row Error

- Ein Skalar-Operator (wie =) kann mathematisch nur mit einem einzigen Wert umgehen.
- Liefert die Subquery durch eine unpräzise Definition plötzlich mehrere Zeilen (z.B. weil **GROUP BY** statt **Aggregation** verwendet wurde), bricht der SQL Server mit einem Fehler ab.
- **Prüfung:** Wenn das Risiko besteht, dass die Unterabfrage eine Liste statt eines Einzelwertes generiert, ist zwingend der Mengenoperator **IN** zu verwenden.

Übung 9
Mengenoperationen & Subqueries

Modul 11

Fortgeschrittene Konzepte

Views

Virtuelle Tabellen für das Reporting

- Ein **View** speichert keinen physischen Datensatz, sondern exklusiv das definierende **T-SQL-Statement** im Systemkatalog.
- Views **abstrahieren** komplexe Join-Ketten zu einem simplen, flachen Objekt für Endanwender oder Reporting-Tools (z.B. PowerBI).

Views

Standardisierung von Berichten

- **Views** erfordern zwingend eindeutige Spaltennamen in der Ergebnismenge (Aliasing ist Pflicht).
- Das **ORDER BY**-Statement ist innerhalb eines Views systemseitig verboten, da relationale Mengen per Definition unsortiert sind. Die Sortierung erfolgt erst beim Abruf des Views.

```sql
CREATE VIEW vw_AktivePatienten AS
SELECT
    p.SVNummer AS Sozialversicherungsnummer,
    p.Nachname AS Familienname,
    s.StationName AS AktuelleStation
FROM Patient AS p
INNER JOIN Station AS s
    ON p.StatID = s.StatID;
```

Common Table Expression (CTE)

Temporäre Result Sets

- Eine **Common Table Expression (WITH-Klausel)** definiert eine **temporäre, benannte Ergebnismenge**, die exklusiv für die Dauer der direkten Folgeabfrage im RAM existiert.

- Sie ersetzen tiefe, fehleranfällige Subquery-Verschachtelungen durch eine strikt **sequentielle Top-Down-Lesbarkeit**.

WITH

Schrittweise Datenvorbereitung

- Mehrere **CTEs** können kommagetrennt **aneinandergereiht** werden. Die nachfolgende Hauptabfrage greift auf die **CTE** zu, als wäre sie eine physische Tabelle.

- Hohe Wartbarkeit, da die **Aggregationslogik vom finalen Join getrennt** wird.

```sql
WITH cte_KostenProPatient AS (
    SELECT
        PatID AS PatientenNummer,
        SUM(Kosten) AS GesamtKosten_EUR
    FROM Behandlung
    GROUP BY PatID
)
SELECT
    p.Nachname AS Familienname,
    c.GesamtKosten_EUR AS Abrechnungssumme
FROM Patient AS p
INNER JOIN cte_KostenProPatient AS c
    ON p.PatID = c.PatientenNummer
WHERE c.GesamtKosten_EUR > 10000.00;
```

Korrelierte Subqueries

Zeilenbasierte Parameterübergabe

- Im Gegensatz zu autarken Subqueries **referenziert** eine **korrelierte Subquery zwingend Attribute der äußeren Hauptabfrage**.
- Die Engine muss die innere Abfrage **iterativ für *jede* generierte Zeile der äußeren Abfrage neu evaluieren**.
- Diese Methodik erzeugt einen **"Row-by-Agonizing-Row"** (RBAR) Verarbeitungszyklus. Sie ist extrem CPU-intensiv und führt bei großen Tabellen unweigerlich zu massiven Performance-Einbrüchen. **Joins** oder **CTEs** sind architektonisch zu bevorzugen.

Modul 11 Stolperfallen

Best Practices

⚠️ Achtung: Der View-in-View Anti-Pattern

- Entwickler tendieren dazu, Views auf Basis anderer Views zu erstellen (Verschachtelung).
- Dies führt zu versteckten Performance-Problemen. Ein scheinbar simples **SELECT * FROM vw_ManagementReport** muss im Hintergrund vom Query Optimizer womöglich 15 tief verschachtelte Tabellen mit Millionen Datensätzen auflösen.
- Regel für saubere Architektur: **Ein View greift direkt auf die Basis-Tabellen zu.** Niemals auf andere Views.

Übung 10
Fortgeschrittene Konzepte

Modul 12

Finales Kursprojekt "AKH-Reporting "

Finales Kursprojekt

AKH-Reporting

- Das finale Projekt transformiert isolierte Einzelabfragen in ein geschlossenes, evidentes Reporting-Set für die Klinikleitung.
- Fokus liegt auf der autonomen Synthese aller DQL-Konzepte: Verknüpfung, Filterung, Aggregation und deterministische Sortierung.

Das Shift-Leader-Dashboard

Die kritischen Management-Fragen

- Erstellen Sie drei robuste T-SQL Skripte, die bei jeder Schichtübergabe fehlerfrei ausgeführt werden müssen.
- **Anforderung 1:** Aktuelle Stationsauslastung (Aggregierte Patientenanzahl pro Station, inkludieren Sie zwingend leere Stationen).
- **Anforderung 2:** Kostenkontrolle (Identifikation der Top 5 teuersten Behandlungen des laufenden Monats).
- **Anforderung 3:** System-Anomalien (Liste aller Patienten ohne zugewiesene Primärdiagnose oder ärztliche Betreuung).

Best Practices: Code-Hygiene

Lesbarkeit für Menschen, nicht für Maschinen

- T-SQL Code wird exakt einmal geschrieben, aber im Lebenszyklus der Applikation hundertfach gelesen, debuggt und gewartet.
- Erzwingen Sie strikte visuelle Einrückungen (Indentation) für SELECT, FROM, WHERE und insbesondere für komplexe JOIN-Bedingungen.
- Kommentare (-- oder /* */) erklären ausschließlich das *Warum* der Business-Logik (z.B. "Filtert Storno-Abrechnungen"), niemals das *Was* der SQL-Syntax.

```sql
-- KORREKT: Lesbare Strukturierung
SELECT
    p.Nachname AS Familienname,
    s.StationName AS ZugewieseneStation
FROM Patient AS p
LEFT JOIN Station AS s
    ON p.StatID = s.StatID
WHERE p.GebDatum >= '2000-01-01';
```

Best Practices: Namenskonventionen

Standardisierung verhindert fatale Fehler

- Tabellen-Aliase müssen semantisch präzise und nachvollziehbar sein (Patient AS p, Behandlung AS beh statt kryptischem a, b, c).
- Result-Set-Spalten via AS erfordern sprechende, betriebswirtschaftlich eindeutige Bezeichnungen (GesamtKosten_EUR statt Summe).
- Konsistenz ist kritischer als der spezifische Case-Style. Mischen Sie niemals CamelCase (PatientenName) mit Snake_Case (patienten_name) im selben System.

Code-Review & Optimierung

Evidenzbasierte Qualitätssicherung im Team

- Syntaktisch fehlerfreier Code bedeutet nicht, dass das betriebswirtschaftliche Ergebnis korrekt ist (z.B. durch Verlust von Datensätzen bei einem falschen INNER JOIN).
- Das Vier-Augen-Prinzip (Peer-Review) deckt logische Trugschlüsse und ineffiziente Filter-Ketten (HAVING statt WHERE) rigoros auf.
- Gemeinsame Refaktorierung: Wir überarbeiten schwer lesbare Verschachtelungen iterativ und ersetzen sie durch modulare CTEs.

Kursabschluss & Systemübergabe

Vom Labor in die Produktion

- Das Fundament der relationalen Algebra und die strikte T-SQL-Syntax sind nun als Handwerkszeug etabliert.
- Weiterführende Skalierung in der Praxis erfordert das Verständnis von physischer Indexierung (B-Trees), Stored Procedures und Zugriffskontrolle (DCL).
- Das AKH-Szenario ist hiermit strukturell und logisch erfolgreich abgewickelt.

Modul 12 Stolperfallen

Best Practices

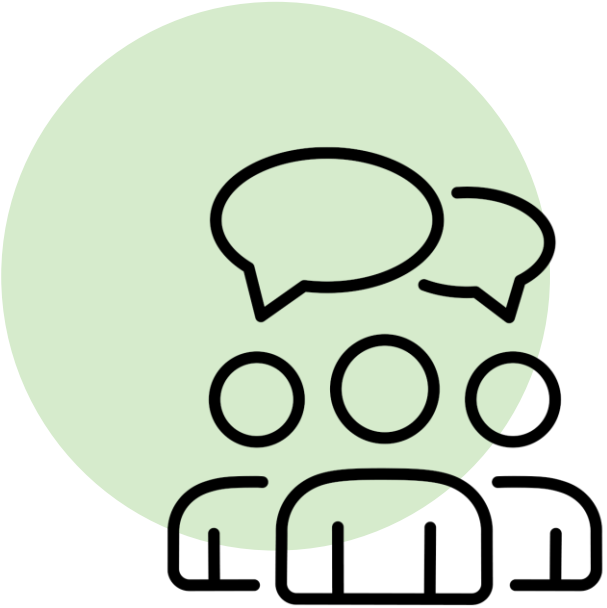
⚠️ Tipp: "Working Code is not Finished Code"

- Geben Sie niemals das erste Skript, das zufällig das richtige Ergebnis liefert, in die Produktion.
- Spaghetti-Code ohne Einrückungen und Aliase wird vom Query Optimizer des SQL Servers zwar problemlos verarbeitet, ist aber für Kollegen unwartbar.

Übung 11
AKH-Reporting

Feedback

Wie geht es mir gerade?

Was fand ich gut?

Was hätte besser laufen können?